Solution Requirements

CrediShield AI — Credit Card Approval Prediction System

---

1. Functional Requirements

ID	Requirement	Priority	Status
FR-01	The system shall accept 14 applicant input fields: gender, age, income, education, marital status, children count, housing type, occupation, years employed, family size, car ownership, property ownership, income type, and employment status.	Must Have	☑ Implemented
FR-02	The system shall predict whether a credit card application is Approved (1) or Rejected (0) using a trained ML classifier.	Must Have	☑ Implemented
FR-03	The system shall return a confidence score between 0% and 100% with every prediction, derived from the model's `predict_proba()` method.	Must Have	☑ Implemented
FR-04	The system shall classify risk as Low (≥85%), Medium (65–84%), or High (<65%) based on confidence thresholds.	Must Have	☑ Implemented
FR-05	The system shall display model comparison metrics (Accuracy, Precision, Recall, F1, ROC AUC) for all 6 trained models in a table format.	Should Have	☑ Implemented
FR-06	The system shall render 4 interactive charts: feature importance (bar), approval ratio (doughnut), income distribution (bar), and age distribution (pie).	Should Have	☑ Implemented
FR-07	The system shall maintain a local prediction history log using browser localStorage, including transaction ID, timestamp, income, decision, and confidence.	Should Have	☑ Implemented
FR-08	The system shall generate and download a printable PDF eligibility assessment report branded as "CrediShield AI Credit Assessment Letter" containing full applicant details and decision.	Should Have	☑ Implemented
FR-09	The system shall support toggling between light and dark themes, with preference persisted in localStorage.	Nice to Have	☑ Implemented
FR-10	The system shall auto-detect missing model files on startup and trigger the training pipeline (`train.py`) automatically.	Nice to Have	☑ Implemented
FR-11	The system shall validate all required input fields and return descriptive error messages (HTTP 400) when fields are missing.	Must Have	☑ Implemented
FR-12	The system shall handle unrecognized categorical values gracefully with descriptive encoding error messages.	Must Have	☑ Implemented
FR-13	The system shall provide a "Clear History" button to purge all stored predictions from localStorage.	Nice to Have	☑ Implemented
FR-14	The system shall allow downloading PDF reports from the history table for any previously stored prediction.	Nice to Have	☑ Implemented

---

2. Non-Functional Requirements

ID	Requirement	Target Metric	Achieved
NFR-01	**Performance**: Prediction response time shall be under 500 milliseconds.	< 500ms	☑ ~85ms achieved
NFR-02	**Usability**: The web dashboard shall load within 3 seconds on a standard broadband connection.	< 3s	☑ Achieved
NFR-03	**Deployability**: The system shall be deployable on free-tier cloud platforms without paid infrastructure.	Render Free Tier	☑ Deployed
NFR-04	**Maintainability**: All code shall be well-commented with section headers, inline comments, and function docstrings.	Every file documented	☑ Achieved
NFR-05	**Responsiveness**: The UI shall be responsive and functional across desktop (1920×1080), laptop (1366×768), and tablet (768×1024) viewports.	3 viewport sizes	☑ CSS media queries applied
NFR-06	**Accuracy**: Model accuracy on test data shall exceed 95%.	> 95%	☑ 98.29% achieved
NFR-07	**Reliability**: The system shall handle invalid inputs gracefully without crashing, returning structured JSON error responses.	Zero unhandled crashes	☑ Try-catch in all endpoints
NFR-08	**Portability**: The system shall run on Windows, macOS, and Linux with Python 3.8+ installed.	Cross-platform	☑ Achieved
NFR-09	**Scalability**: The architecture shall support future migration to PostgreSQL for persistent storage.	Modular design	☑ Endpoints are decoupled
NFR-10	**Security**: The system shall not expose raw model internals or stack traces in API error responses.	No sensitive data leaks	☑ Generic error messages

---

3. Data Requirements

Requirement	Details
Primary Dataset	`application_record.csv` — 438,510 rows × 18 columns containing applicant demographic and financial data
Secondary Dataset	`credit_record.csv` — 1,048,575 rows × 3 columns containing monthly payment status records
Join Key	`ID` column present in both datasets
Target Engineering	Binary labels derived from payment status codes (Safe → Approved, Delinquent → Rejected)
Missing Data Handling	OCCUPATION_TYPE nulls filled with 'Unknown'
Outlier Removal	Income outliers filtered using IQR method
Train-Test Split	80/20 stratified split to preserve class ratio

---

4. Interface Requirements

4.1 User Interface (Frontend)

- Single-page application with 4 tabbed sections (Predict, Metrics, Analytics, History)
- Glassmorphism obsidian dark theme with frosted glass panels
- Animated SVG circular progress gauge for confidence display
- Chart.js integration for interactive data visualizations
- Responsive design supporting desktop, laptop, and tablet viewports
- Google Fonts (Plus Jakarta Sans) for premium typography

4.2 API Interface (Backend)

Endpoint	Method	Request	Response
`/`	GET	—	HTML dashboard page
`/predict`	POST	JSON with 14 fields	`{approved, confidence, risk_level}`
`/metrics`	GET	—	`{metrics: {...}, stats: {...}}`

---

5. Constraint Summary

Constraint Type	Description
Budget	$0 — all tools and platforms must be free-tier
Timeline	12-day development cycle
Team Size	1 developer (solo project)
Runtime	Python 3.12, Gunicorn 22.0.0
Hosting	Render.com Free Tier (512MB RAM, cold starts after 15 min)
File Size	GitHub 100MB limit per file; model artifacts < 150KB total
Bundle Size	Vercel 500MB limit (exceeded → switched to Render)

Project Name: CrediShield AI
Author: Dinesh (dineshTech-creator)
Date: July 2026